Mechanisms of Action Prediction — Kaggle

Machine Learning

Professors:
Dirk VALKENBORG
Inigo BERMEJO DELGADO

Eleftherios Kokkinis
Racheal Natumanya
Parisa Farrokhian
Soumaya Kasri

December 17, 2024

Contents

1 Introduction **3**

2 Data Description **3**

3 Multi-label classification **3**

4 Existing approaches **4**

5 MLC Exploratory Metrics **4**
- 5.1 Assessing multilabelness . . . 4
  - 5.1.1 Cardinality . . . 4
  - 5.1.2 Density . . . 5
  - 5.1.3 P_{min} metric . . . 5
- 5.2 Assessing Imbalance . . . 5
  - 5.2.1 Imbalance Ratio for Label . . . 5
  - 5.2.2 Maximum and Mean Imbalance Ratio . . . 6
- 5.3 Assessing concurrence levels . . . 6

6 Exploratory Data Analysis **7**

7 Evaluation Metric **9**

8 Modeling **9**
- 8.1 Baseline XGBoost . . . 9
- 8.2 OVR approach . . . 9
- 8.3 Ensemble of Classifier Chains . . . 10

9 Resampling with MLSMOTE **11**

10 Dimensionality Reduction **12**

11 Enhanced modeling **14**

12 Future Work **14**

13 Appendix **16**

1 Introduction

The Mechanism of Action of a drug describes the specific biochemical interactions through which the drug achieves its pharmacological effect. In essence, it explains the molecular or cellular processes that underlie the drug's therapeutic impact on the body. A thorough understanding of MoA is critical for healthcare professionals, as it sheds light on how a drug interacts with biological systems, informs its clinical applications, and guides the development of effective treatment strategies.

This challenge revolves around predicting the biological effects of drugs. Each drug can simultaneously influence multiple mechanisms of action, highlighting the complex interplay of molecular pathways and cellular processes. This is something that underscores the importance of accurate MoA prediction. Identifying the MoA of a drug not only facilitates the discovery of therapeutic targets but also enhances the efficiency of drug discovery efforts and reduces reliance on chance findings.

In this challenge we seek to classify drugs based on their biological activities.

2 Data Description

Our challenge's dataset is designed to predict the mechanism of action of drug compounds based on molecular and experimental data. It provides a set of features describing various aspects of the compounds, including gene expression levels, compound type, dose, and cell line, alongside binary columns representing the presence or absence of potential MoAs.

The obvious issue here is the high-dimensional nature which arises from the inclusion of gene expression data, numerous potential MoA labels, and categorical variables. With over 1000 features and each compound potentially exhibiting multiple mechanisms of action, the task becomes one of multi-label classification.

In this case, for each compound, the dataset provides a series of binary labels for different MoAs. A label of "1" indicates that the compound exhibits the given MoA, and a label of "0" means the MoA is not present. Since a single compound may activate more than one MoA, this makes the problem inherently multi-label, where the model must predict multiple binary outcomes simultaneously.

The features contain molecular and experimental data on each compound and include the following :

sig id: A unique identifier for each compound or drug.

cp type: Type of compound (e.g., "trt cp" for treatments or "ctl gp" for control groups).

cp dose: The dose of the drug (D1 for the higher dose and D2 for the lower dose).

gene expression: This represents the expression levels of various genes that can help determine the molecular effects of the drug.

cell line: The type of cell line used in the experiment. Different cell lines can have varying responses to drugs.

MoA columns: Each column corresponds to a different mechanism of action, and the values are binary, where 1 indicates the presence of the mechanism of action for that compound, and 0 indicates its absence.

3 Multi-label classification

Considering the structure of our data we conclude that we are dealing with a multi-label classification problem. Each one of our data instances has associated a vector of outputs - the MoAs, instead of only one value. The length of our vector is fixed to 206, that is the number of different Mechanisms of Action in our dataset. Each element of the vector is a binary value, indicating if the corresponding MoA is relevant to the sample or not. Several labels can be active at once and each distinct combination of labels is known as labelset [1] .

We can conclude that the main difference between traditional and our multilabel classification problem

is the output expected the trained model. Where a traditional classifier would return only one value, we have to produce a vector of output values.

4 Existing approaches

Currently, in multilabel classification tasks, diverse strategies are applied, such as transforming the data into a suitable format and adapting traditional classification methodologies. The use of ensemble classifiers is a prevalent approach as well. Moreover, specific challenges like capturing label dependency, addressing high dimensionality, and dealing with label imbalance are critical considerations.

In our case, the adaptation of multilabel classification builds upon previous work in related domains, such as protein classification, where structural motifs and patterns like short amino acid chains have been effectively used as predictive features [2], [3]. Similarly, the MoA problem leverages high-dimensional biological data to uncover patterns that correlate with the multiple responses a single drug can induce

5 MLC Exploratory Metrics

Our multilabel predictive performance evaluation metrics have to deal with the presence of multiple outputs, which means also taking into consideration the existence of predictions which are partially correct or wrong.

We bring as a an example another publicly available ML dataset, the yeast dataset, where the goal was to predict the functional expression for a set of genes. The input features for each gene come from microarray expression data, with a 103 real values vector per instance. A subset of 14 functional classes, whose origin is the Comprehensive Yeast Genome Database,[1] are selected and used as labels. Since each gene can express more than one function at once, the result is also a dataset with multi-label nature.

Before we proceed to define a classification model for our own task, it is rather essential to delve into the inner traits characteristics the data that we have available to tackle the problem. By understanding these traits we will also become able to select the best possible algorithm, parameters etc.

5.1 Assessing multilabelness

5.1.1 Cardinality

First, we would like to assess the extent at which the samples in our dataset have more than one label. We achieved that by counting the number of labels relevant to every instance in the dataset, then averaging the sum to know the mean number of labels per instance.

Cardinality of D:

$$\mathrm{Card}(D) = \frac{1}{n}\sum_{i=1}^{n} |Y_i|$$

n denotes the number of instances in our dataset (D), While Y_1 is the labelset of the ith instance. The higher our card level, the larger is the number of active labels per instance.
It was found that we have a cardinality of **0.7**, which means that on average, each sample is associated with 0.7 MoAs. That is even less than 1 (!), which means that each instance is associated with 1 or no labels. That could denote that we have a dataset with little multilabeleness nature. However, cardinality is a metric that is influenced by the size of the set of labels. Which implies that our dataset may just be sparse, and most samples have either 0 or 1 relevant label. However, this does not mean the problem is not multi-label but that we may just have low density,

[1] http://www.ncbi.nlm.nih.gov/pubmed/15608217

5.1.2 Density

Density accounts for the label set size and is a relative measure of multilabelness. By dividing our cardinality by the number of labels - in our case 206 -, we can obtain a dimensionless metric, the label density. A higher density would indicate that the labels in the dataset are well represented in each instance, while a low density denotes that only a small subset of labels is present in most instances.

Density of D:

$$\text{Dens}(D) = \frac{1}{k} \cdot \frac{1}{n} \sum_{i=1}^{n} |Y_i|$$

n again denotes the number of instances, D our dataset, and k the total number of labels in D. While Y_i is the labelset of the ith instance.
It was found that we have density **0.0034**, which means only 0.34% of the labels are active across the dataset! In other words only 0.34% of our MoAs become active for each drug. That confirms the low sparsity of the labels.

5.1.3 P_{min} metric

Since cardinality can be a a misleading metric in our case, we will use P_{min} to complete our understanding of the multi label nature of our data. This metric is simply the percentage of instances with at least one / or more than 1 (depending on how we define it each time) active label.

$$P_{\min}(D) = \frac{\sum_{y \in Y} I(|y| \geq 1)}{n}$$

In this definition we get the percentage of instances where at least 1 MoA is active, and in our case that was 60%. (40% of our data have no active MoAs, in that percentage the control samples are included as well).

In the following definition, we get the percentage of instances where more than 1 MoA is active, and that percentage is 8%.

$$P_{\min}(D) = \frac{\sum_{y \in Y} I(|y| > 1)}{n}$$

A low P_{min} in that case means that only a small number of instances has more than one active MoA.

5.2 Assessing Imbalance

Taking into account that we are dealing with a classification problem, the presence of class imbalance can be a challenge for most learning algorithms. Although we have not conducted an exploratory analysis yet, we know that most real multi-label datasets usually suffer from class imbalance [1]. Which means that some labels show higher frequency than others. We will make use of three different metrics that are proposed in [4] to assess label imbalance, named IRLlbl, MaxIR and MeanIR, before we even attempt to assess it visually.

5.2.1 Imbalance Ratio for Label

$$\text{IRLbl}(l) = \frac{\max_{l \in L} \sum_{i=1}^{n} I[l \in Y_i]}{\sum_{i=1}^{n} I[l \in Y_i]}$$

This metric allows us to assess the imbalance level of one specific label. It is the proportion between the number of appearances of the most common label and our label. So of course if we assess the most commonn label we would have IRLbl = 1, while for least frequent labels the level is always greater.

5.2.2 Maximum and Mean Imbalance Ratio

With MAxIR we are asessing the proportion of the most common label against the most rare one while with MeanIR we aim to assess the global imbalance by basically averaging the IRLbl of all labels

$$\text{MaxIR} = \max_{l \in L}(\text{IRLbl}(l))$$

$$\text{MeanIR} = \frac{1}{|L|} \sum_{l \in L} \text{IRLbl}(l)$$

It was found that MaxIR = 832.0 and MeanIR = 41.23. A high MaxIR and MeanIR would indicate that the rare labels are underrepresented and thus lead to poor generalization.

Specifically, MaxIR = 832.0 means that the most frequent label occurs 832 times more often than the least frequent, while MeanIR = 41.24 means that on average, the imbalance ratio across all labels is 41.24.
However, a high MeanIR could also be due to a relatively high IRLbl for several labels or by the extreme imbalance for only some labels. So we also utilize the coefficient of variation for imbalance ratios which quantifies the variability of imbalance ratios across the labels.
This is defined as the standard deviation of the imbalance ratios IRLbl(l) across all labels divided by the Mean of the imbalance ratios.
In that way we can see how unevenly distributed the imbalance ratios are across labels. A higher CVIR would mean that the ratios vary significantly, sosome labels are extremely underrepresented, while others are relatively common.

$$\text{CVIR} = \frac{\sigma(\text{IRLbl})}{\text{MeanIR}}$$

$$\sigma(\text{IRLbl}) = \sqrt{\frac{1}{|L|-1} \sum_{l \in L} (\text{IRLbl}(l) - \text{MeanIR})^2}$$

We found that $CVIR = 2.11$ which is a quite substantial variation in the imbalance across labels. We can say that some labels are much rarer than others, and the model might struggle to learn about these labels without intervention.

5.3 Assessing concurrence levels

As another exploratory step, it would be useful for us to see to what extend there is concurrence among very frequent and rare labels. We made use of the SCUMulative BLEvel metric [5] -another proposed metric to measure label imbalance-. It essentially quantifies how imbalanced each instance is considering all its active labels.

$$\text{SCUMBLE}_{\text{ins}}(i) = 1 - \frac{1}{\text{IRLbl}_i} \left(\prod_{l \in L} \text{IRLbl}_i^l \right)^{1/k}$$

Using the same notations as before, for each instance i, we let L_i be the set of active labels and k the total number of labels. The metric takes values in the [0, 1] range indicating if all the labels in the instance have similar frequencies in the dataset. Of course in the instance-specific formula, if an instance doesn't have any active labels, then SCUMBLE would be zero.
Since we are interested in a global assessment, we will obtain the global SCUMBLE by averaging all the scumble values for all instances in the dataset.

$$\text{SCUMBLE}(D) = \frac{1}{n} \sum_{i=1}^{n} \text{SCUMBLE}_{\text{ins}}(i)$$

A higher value could mean that instances often contain rare labels, making the dataset more challenging while a lower scumble would mean that instances have more common labels on average.

It was found that $SCUMBLE = 0.06$. We could thus say that the instances of our data are rather moderately imbalanced on average, with the scumble metric falling slightly below 0.5 - we could even claim that the instances tend to be composed of more common labels.

6 Exploratory Data Analysis

Table 1: Dataset Statistics

num.attributes	num.instances	num.inputs	num.labels	num.labelsets	num.single.labelsets	max.frequency	cardinality
1083	23814	877	206	328	6	9367	0.707315
density	**meanIR**	**scumble**					
0.003434	41.23725	0.006701					

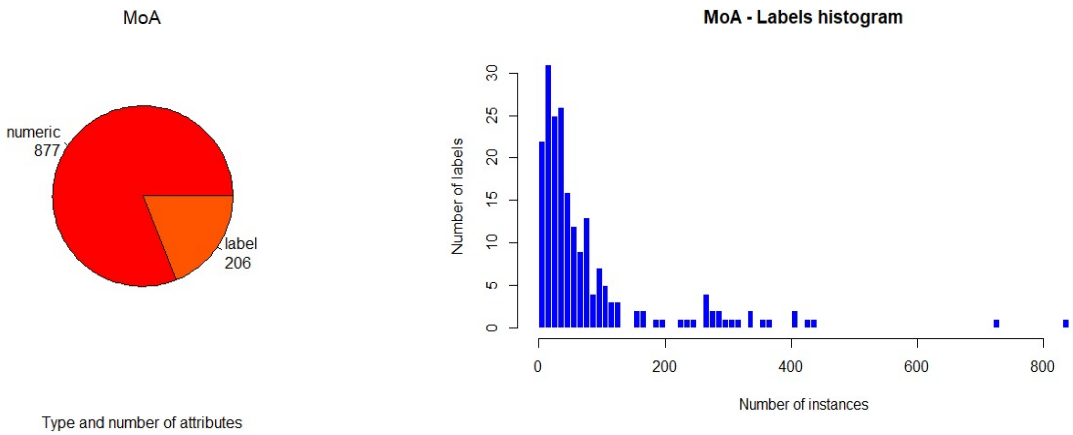

Figure 1: Overview of dataset attributes and label distribution.

The label histogram shows us how well-represented labels are in general. The X-axis represents the number of instances, and the Y-axis represents the number of labels. We note that many labels appear in very few instances, thus the data concentrates on the left side of the plot. Many labels are therefore under-represented, which translates to class imbalance.

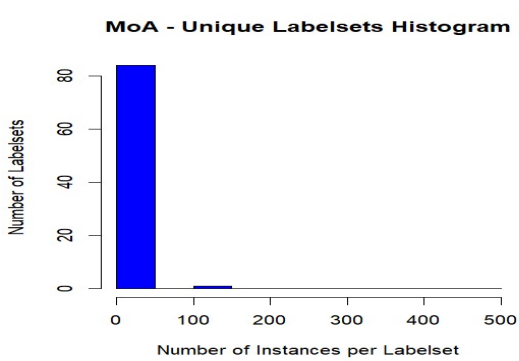

Figure 2: Unique label sets

The labelset histogram shows the amount of labelsets instead of the number of instances per label. The dataset has a large number of rare labelsets, where most combinations of labels appear only a few times. A small number of labelsets on the other hand seem to appear more frequently, dominating the dataset.

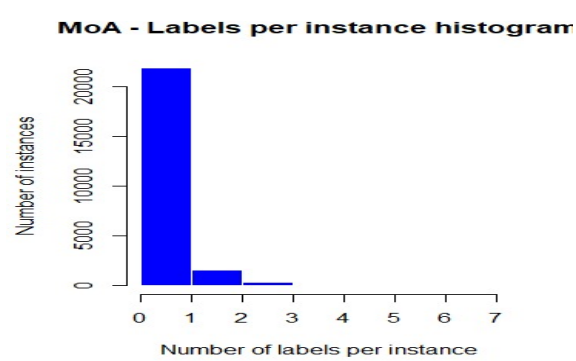

Figure 3: Labels per instance

The cardinality histogram represents the number of labels instances have. We can see that data are accumulating on the left which indicates fewer labels per instance. This aligns with the previously calculated cardinality metric.

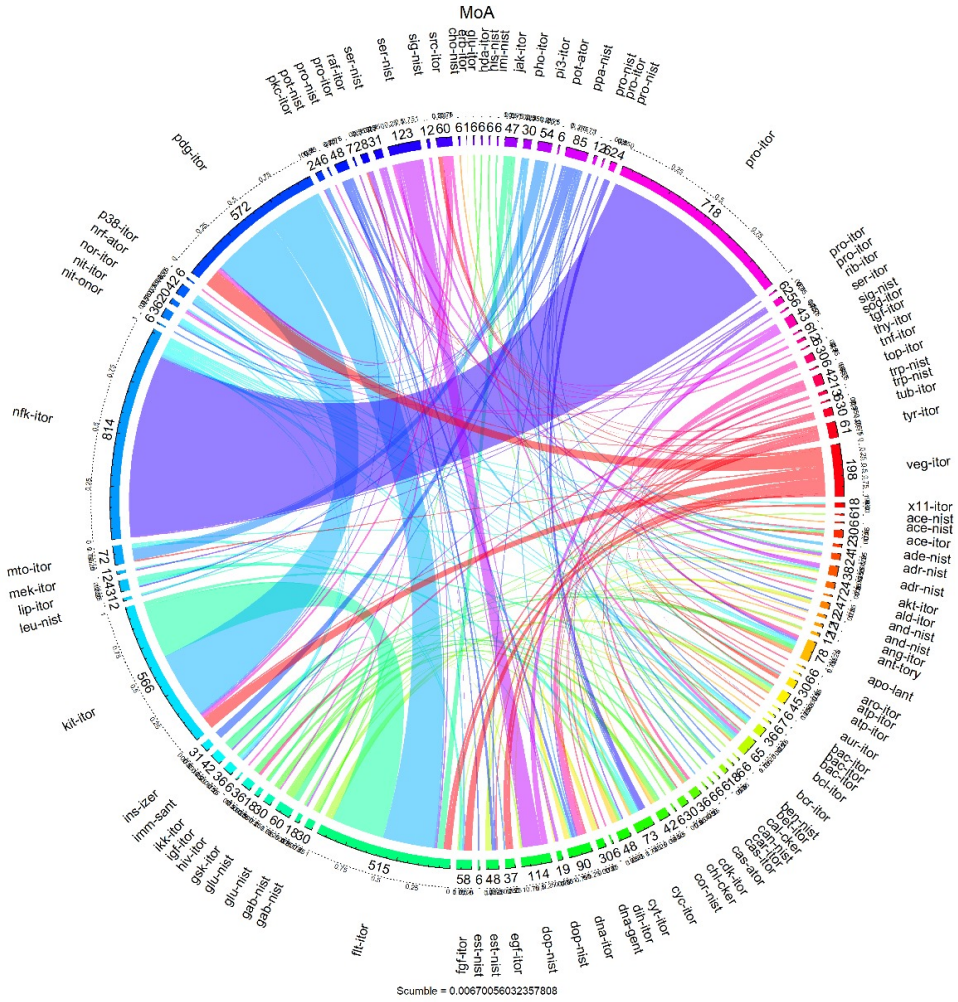

Figure 4: Mechanism of Action counts

The concurrence plot responds to our need of exploring interactions among labels, and specifically between majority and minority ones. The disjoint arcs here are representing labels. Each arc has length proportional to the number of instances where the label is present. The arcs are in turn divided into bands that join two of them, showing the relation between the corresponding labels. The width of each band indicates the strength of the relation, since it is proportional to the number of instances in which both labels appear simultaneously. In this manner, Figure 4 shows whether imbalanced labels appear frequently together, a situation which could limit the possible improvement of applying a resampling technique. We saw previously that our data's scumble value was quite low, which means we do not suffer from this issue.

That's further validated from the plot. MoAs with small arcs also tend to have low width connections, whilst MoAs with larger across have wider connections.

7 Evaluation Metric

Considering we are working on a competition problem, our primary goal is merely to maximize our score. We have already been handed the evaluation function upon which our model will be evaluated, that is the Log Loss function and thus our goal is its minimization. Specifically, the evaluation function was defined as follows:

For every *sig_id*, we predict the probability that the sample had a positive response for each MoA target. Given:

- N: The number of *sig_id* observations in the test data ($i = 1, \ldots, N$)
- M: The number of scored MoA targets ($m = 1, \ldots, M$)
- $\hat{y}_{i,m}$: The predicted probability of a positive MoA response for sig_id_i and target m
- $y_{i,m}$: The ground truth binary label for sig_id_i and target m (1 for positive response, 0 otherwise)

Log loss:

$$\text{score} = -\frac{1}{M}\sum_{m=1}^{M}\frac{1}{N}\sum_{i=1}^{N}\big[y_{i,m}\log(\hat{y}_{i,m}) + (1-y_{i,m})\log(1-\hat{y}_{i,m})\big].$$

Since the function is undefined for probabilities of 0 and 1, the submitted predicted probabilities are clipped:

$$\hat{y}_{i,m} \leftarrow \max\big(\min(\hat{y}_{i,m}, 1-10^{-15}),\, 10^{-15}\big).$$

A smaller log loss indicates better performance.

8 Modeling

8.1 Baseline XGBoost

In order to set a baseline for our incremental modeling enhancements, we trained a simple XGBoost model using a MultiOutputClassifier, alongside the log loss function.
Categorical variables like time and dose were one hot encoded and then removed from the dataset. The model's parameters were optimized with the optuna framework on a validation test with a 80/20 split. The model was then retrained on the whole dataset and benchmarked on Kaggle with the following results.

Private Score	**Public Score**
0.02607	0.02932

Table 2: Comparison of Private and Public Scores

8.2 OVR approach

The "One vs the rest" approach, is one of the easiest ways to tackle our multilabel classification problem [6]. That would entail to take one label at a time and train individual classifiers for each one of them, basically having an ensemble of bianry classifiers. Their outputs are then combined to generate the predicted labelset, an approach known in the literature as Binary Relevance [7]. We implemented the BR using Python's scikit-multilearn framework [8]. After we completed our training, we used the test samples as input to each binary classifier and then joined the individual outputs to form the required labelset as a result. For that approach we used XGBoost as the underlying tree-based classifier.

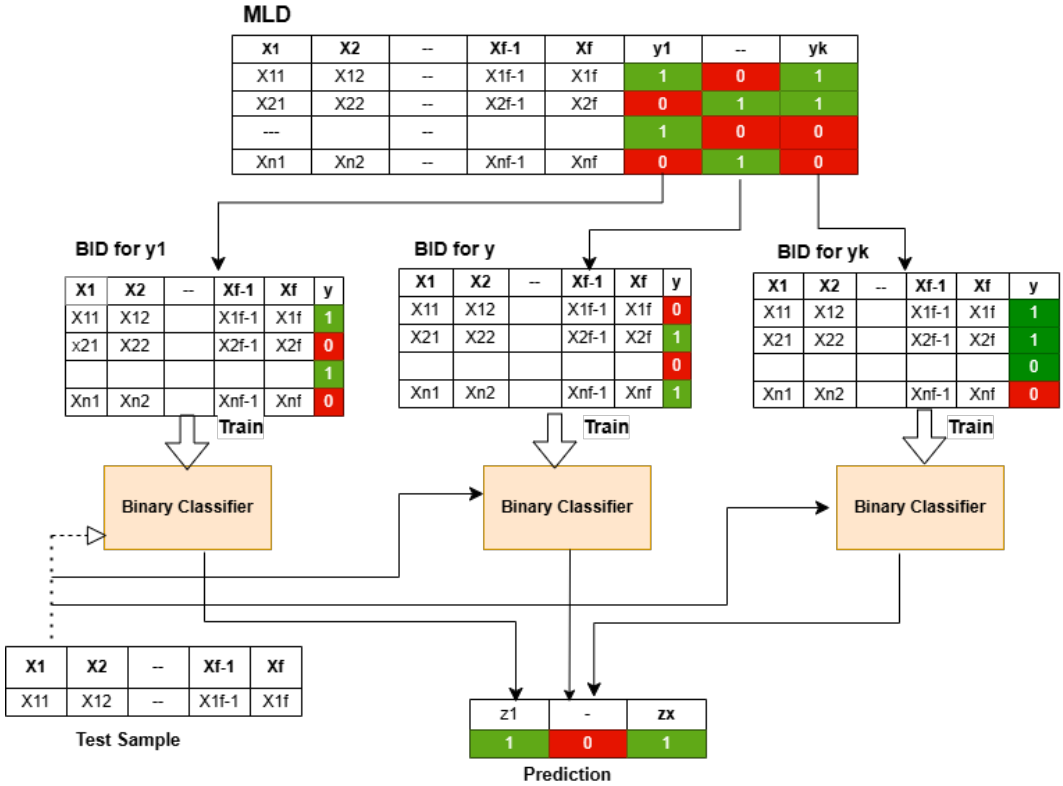

Figure 5

Figure 6 illustrates the BR approach, while using XGBoost as a binary classifier.
The aforementioned approach was carried out again with a simple 80/20 train test split, without CV, dimensionality reduction, resampling or other preprocessing steps aimed at alleviating some of the issues inherent to our data. That was a conscious decision in order to see how incremental changes and enhancements can affect our model's performance.
A noteworthy improvement in the Log Loss was observed when using BR with XGboost:

Private Score	Public Score
0.01783	0.02059

Table 3: Comparison of Private and Public Scores

8.3 Ensemble of Classifier Chains

A noteworthy drawback of the BR approach is that since we're training independent classifiers, we dismiss any potential correlation among the labels as well. We will thus proceed testing out classifier chains [9]. In that approach we trained the first classifier only on the initial features, and then added the output label as new input feature to train the second classifier and so on. That's offering us a way to capture possible label dependencies. Considering however that the order of the labels affect the results, we opted for the ECC adaptation, that is an Ensemble of Classifier Chains that goes a step further by training multiple CC models with different chain orders and averaging their predictions.
That was an extremely expensive process, resulting to a model of over 460MB.
For the sake of time no optimization was carried out. XGBoost was again utilized as before and used the same hyperparameters that were found from the BR method. Eventually, due to the very high time complexity of the algorithm and the limited resources available, this method was dropped, with high interest of re-testing it

after the feature space is reduced.

9 Resampling with MLSMOTE

Considering our previous findings on class imbalance and how that challenges any model on making accurate predictions for these classes, we utilize the multi label adaptation of SMOTE. We're basically generating synthetic samples by interpolating feature values between an instance and its nearest neighbors. The algorithm is focusing on minority labels which we defined as labels with IR higher than the mean IR, and it considers label dependencies when generating new samples.

In summary our implementation proceeds as follows:

- We Identify Minority Labels: Calculate the Imbalance Ratio for each label. If a label's IR exceeds the Mean IR, it is considered a minority label.
- Selecting Minority Instances: Extracting instances where the minority label is active.
- Finding Nearest Neighbors: For each minority instance, we find its k nearest neighbors, we used k = 5.
- Generating Synthetic data: By interpolating the values between the original instance and a randomly chosen neighbor.

In the end of the process, the algorithm created 609 new samples. By calculating the mean IR again, we noticed a significant drop from 41.23 to 31.10.

By applying the MLSMOTE algorithm consecutively we were able to decrease the mean IR to 21.29, about half of its original. However at that value the imbalance ratio plateaued and no further reduction could be achieved.
Although by no means do we claim that the imbalance issue is now fully resolved, we have taken a step towards alleviating issue the, with measurable results. Due to the constraint of time and for the sake of simplicity of the analysis no further action on generating synthetic data was taken.

Table 4: Dataset Statistics after 1st MLSMOTE

num.attributes	**num.instances**	**num.inputs**	**num.labels**	**num.labelsets**	**num.single.labelsets**	**max.frequency**
1083	24423	877	206	328	6	9367
cardinality	**density**	**meanIR**	**scumble**			
0.7215739	0.003502786	31.10358	0.006920091			

Table 5: Dataset Statistics after 2ns MLSMOTE

num.attributes	**num.instances**	**num.inputs**	**num.labels**	**num.labelsets**	**num.single.labelsets**	**max.frequency**
1084	25323	878	206	328	6	9367
cardinality	**density**	**meanIR**	**scumble**			
0.7341152	0.003563666	25.45756	0.006679384			

Table 6: Dataset Statistics after 3rd MLSMOTE

num.attributes	**num.instances**	**num.inputs**	**num.labels**	**num.labelsets**	**num.single.labelsets**	**max.frequency**
1083	26154	877	206	328	6	9367
cardinality	**density**	**meanIR**	**scumble**			
0.7515103	0.003648108	23.32258	0.006663828			

Table 7: Dataset Statistics after 4th MLSMOTE

num.attributes	num.instances	num.inputs	num.labels	num.labelsets	num.single.labelsets	max.frequency
1082	27391	876	206	330	7	9367
cardinality	**density**	**meanIR**	**scumble**			
0.7709467	0.00374246	21.29361	0.0064351			

10 Dimensionality Reduction

From the data description and our exploratory data analysis it was already observed that we are dealing with several hundreds of features. Unlike regular single label classification problems, we are also dealing with a high dimensional output space, so determining the significance of every input attribute to each joint labelset is not an easy job. The time complexity of the binarization techniques applied in the previous sections increases linearly with respect to the number of labels, which in turn made the training of our models computationally expensive. In addition, as the set of the labels grows, the number of combinations increases exponentially, resulting to only a few samples as representatives of each labelset. Consequently this sparseness degrades the performance of our model as well.

Therefore our next step will be to reduce the output space, which could contribute to a decrease in time and memory needed but also could result to a simpler classifier that could generalize better.
We note that supervised techniques take advantage of class labels and can analyze correlations between attributes and classes [10] [11], determining the dependencies between inputs and outputs by means of cross variance matrices.

One of these options is Linear Discriminant Analysis. However that's a technique that cannot be directly used to multi label datasets.
To tackle this obstacle we will use Label powerset [12], which means transforming our multi label dataset into a multi class dataset, by taking each label combination as class identifier. The process is showcased on Figure 7. Then we apply the LDA algorithm.

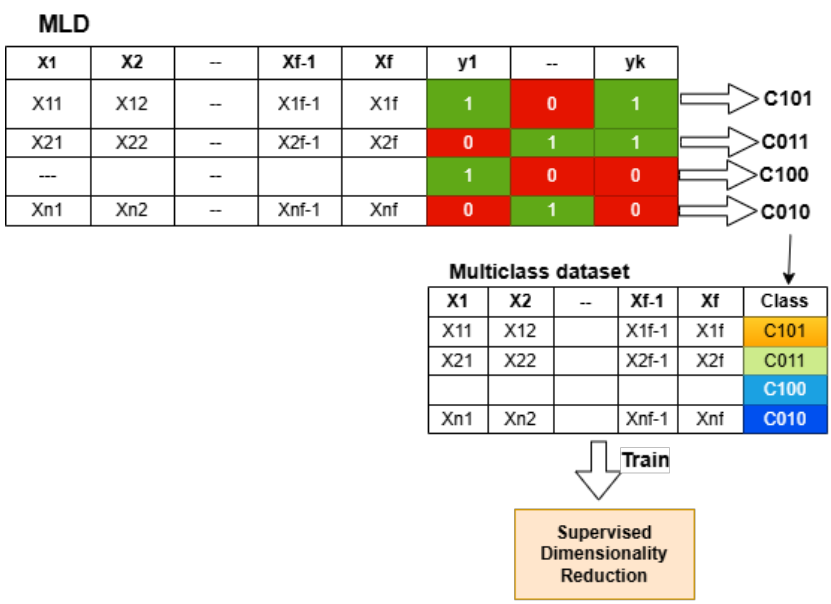

Figure 6

After applying LabelPowerset to transform the multi-label problem into a multi-class problem, we first use the Information Gain as an extra pre-processing step to assess the importance of features on the labels. It basically quantifies how much "information" a feature gives us about the target variable. A higher IG score for a feature means that feature is more informative for predicting the label.

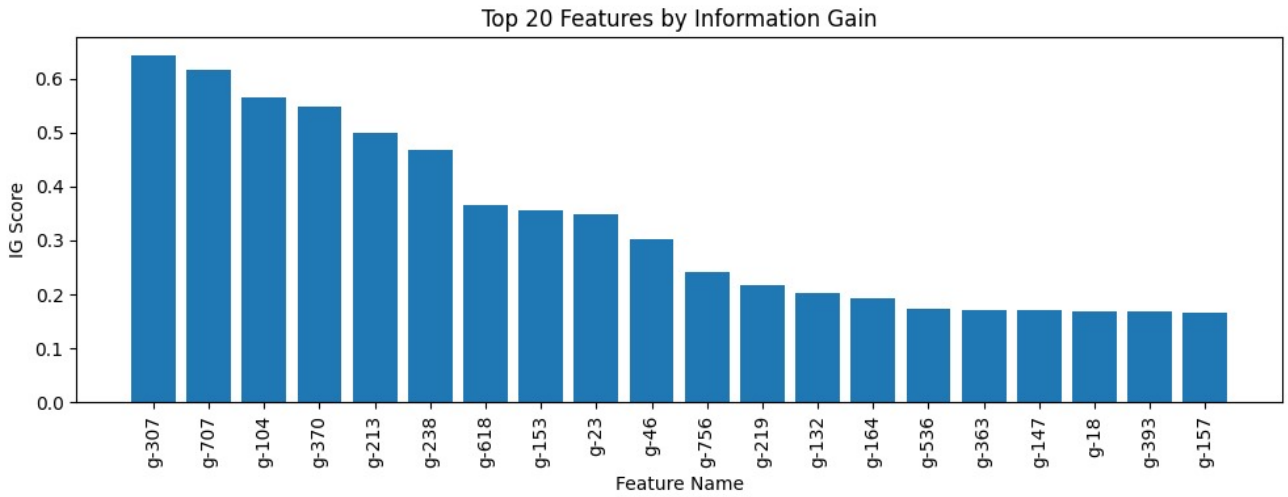

Figure 7

So after calculating the IGs, we picked the top 100 as an extra feature selection step before applying LDA. We note that some of the most informative features are g-307, g-707, g-104, and g-370, among others.

Then LDA projects the data into a lower-dimensional space that maximizes class separability. Figure 8 below shows how samples cluster according to their LP classes in the reduced space.

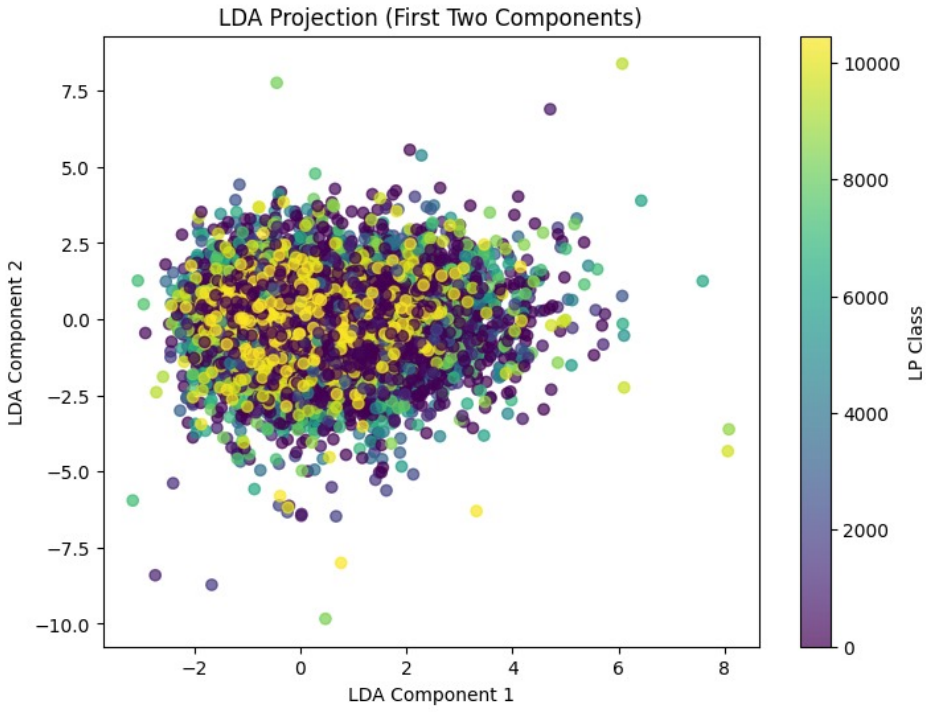

Figure 8

11 Enhanced modeling

To sum up, in the previous sections we tackled the imbalanced problem with resampling, we reduced the issue of high dimensionality by applying both feature selection and dimensionality reduction with the Information Gain and LDA, Then we re-trained a BR of XGboost, using a 5-fold cross validation for more reliable predictions.

We note that no hyperparameter optimization was carried out, which is naturally going to improve our performance. 0verall, another significant drop in the log loss accuracy was achieved.

Private Score	**Public Score**
0.01673	0.01772

Table 8: Comparison of Private and Public Scores

12 Future Work

A large amount of possibilities for enhancements and optimizations have been omitted for the sake of time.

The current analysis however offers a lot of room for further exploration. Now that our dimensional space is more manageable, re-training an ensemble chain of classifiers could be tested again. Other proposed algorithms like Rank-SVM [12], ML-Tree or Ensemble of Prune Sets [13] have shown promising results as well. The implementation of some of these algorithms has to be made from scratch as publicly available packages were not available. In addition python's API for Weka.
Feature selection and dimensionality reduction methods can be further explored as well, with the possibility of feature extraction as well. Finally, several models underwent no optimization, conservative results are expected.

References

[1] F. Herrera, F. Charte, A. J. Rivera, and M. J. del Jesús, *Multilabel Classification: Problem Analysis, Metrics and Techniques*. Switzerland: Springer, 2016, ISBN: 978-3-319-41110-1. DOI: 10.1007/978-3-319-41111-8. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-41111-8.

[2] S. Diplaris, G. Tsoumakas, P. Mitkas, and I. Vlahavas, "Protein classification with multiple algorithms," in *Proc. 10th Panhellenic Conference on Informatics, PCI'05*, vol. 3746, Springer, 2005, pp. 448–456.

[3] A. Elisseeff and J. Weston, "A kernel method for multi-labelled classification," in *Advances in Neural Information Processing Systems*, vol. 14, MIT Press, 2001, pp. 681–687.

[4] F. Charte, A. J. Rivera, M. J. del Jesus, and F. Herrera, "Addressing imbalance in multilabel classification: Measures and random resampling algorithms," *Neurocomputing*, vol. 163, pp. 3–16, 2015.

[5] F. Charte, A. J. Rivera, M. J. del Jesus, and F. Herrera, "Concurrence among imbalanced labels and its influence on multilabel resampling algorithms," in *Proceedings of the 9th International Conference on Hybrid Artificial Intelligent Systems (HAIS'14)*, ser. Lecture Notes in Computer Science, vol. 8480, Springer, 2014, pp. 110–123. DOI: 10.1007/978-3-319-07617-1_10.

[6] M. Galar, A. Fernández, E. Barrenechea, H. Bustince, and F. Herrera, "An overview of ensemble methods for binary classifiers in multi-class problems: Experimental study on one-vs-one and one-vs-all schemes," *Pattern Recognition*, vol. 44, no. 8, pp. 1761–1776, 2011. DOI: 10.1016/j.patcog.2011.01.017.

[7] S. Godbole and S. Sarawagi, "Discriminative methods for multi-labeled classification," in *Advances in Knowledge Discovery and Data Mining*, ser. Lecture Notes in Computer Science, vol. 3056, Springer, 2004, pp. 22–30. DOI: 10.1007/978-3-540-24775-3_3.

[8] P. Szymański and T. Kajdanowicz, "A scikit-based Python environment for performing multi-label classification," *ArXiv e-prints*, Feb. 2017. arXiv: 1702.01460 [cs.LG].

[9] J. Read, B. Pfahringer, G. Holmes, and E. Frank, "Classifier chains for multi-label classification," *Machine Learning*, vol. 85, pp. 333–359, 2011. DOI: 10.1007/s10994-011-5256-5.

[10] H. Hotelling, "Relations between two sets of variates," in *Breakthroughs in Statistics*, Springer, 1992, pp. 162–190.

[11] R. A. Fisher, "The statistical utilization of multiple measurements," *Annals of Eugenics*, vol. 8, no. 4, pp. 376–386, 1938.

[12] M. Boutell, J. Luo, X. Shen, and C. Brown, "Learning multi-label scene classification," *Pattern Recognition*, vol. 37, no. 9, pp. 1757–1771, 2004.

[13] J. Read, B. Pfahringer, and G. Holmes, "Multi-label classification using ensembles of pruned sets," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM'08)*, IEEE, 2008, pp. 995–1000.

13 Appendix

```python
#### BR WITH XGBOOST #####
!pip install scikit-multilearn xgboost optuna

import pandas as pd
import numpy as np
import xgboost as xgb
from sklearn.model_selection import train_test_split
from skmultilearn.problem_transform import BinaryRelevance
from xgboost import XGBClassifier
from sklearn.metrics import log_loss
import optuna
import sys

train_features = pd.read_csv('/root/.cache/kagglehub/competitions/lish-moa/train_features.csv', index_col=0)
train_targets_scored = pd.read_csv('/root/.cache/kagglehub/competitions/lish-moa/train_targets_scored.csv', index_col=0)
test_features = pd.read_csv('/root/.cache/kagglehub/competitions/lish-moa/test_features.csv', index_col=0)
sample_submission = pd.read_csv('/root/.cache/kagglehub/competitions/lish-moa/sample_submission.csv', index_col=0)

MoA_columns = train_targets_scored.columns.tolist()

def preprocess_features(df):

    df['cp_dose_0'] = (df['cp_dose']=='D1').astype(int)
    df['cp_dose_1'] = (df['cp_dose']=='D2').astype(int)

    df['cp_time_24'] = (df['cp_time']==24).astype(int)
    df['cp_time_48'] = (df['cp_time']==48).astype(int)
    df['cp_time_72'] = (df['cp_time']==72).astype(int)

    df = df.drop(['cp_time','cp_dose','cp_type'], axis=1, errors='ignore')
    return df

train_features = preprocess_features(train_features)
test_features = preprocess_features(test_features)

X = train_features.values
Y = train_targets_scored.values


X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.2, random_state=42)
print(f"X_train: {X_train.shape}, X_val: {X_val.shape}, Y_train: {Y_train.shape}, Y_val: {Y_val.shape}")

def objective(trial):
    param = {
        'use_label_encoder': False,
        'eval_metric': 'logloss',
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 1e-3, 0.3, log=True),
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'device':'cuda'
```

```
    }


    br_classifier = BinaryRelevance(
        classifier=XGBClassifier(**param),
        require_dense=[False, True]
    )

    br_classifier.fit(X_train, Y_train)


    val_probs = []
    for i, clf in enumerate(br_classifier.classifiers_):

        p = clf.predict_proba(X_val)[:,1]
        val_probs.append(p)
    val_probs = np.column_stack(val_probs)

    score = 0.0
    for i in range(Y_val.shape[1]):
        score += log_loss(Y_val[:, i], val_probs[:, i], labels=[0,1])
    score /= Y_val.shape[1]

    return score

print("Starting hyperparameter tuning with Optuna...")
study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=10)

best_params = study.best_params
print("Best Params:", best_params)
print("Best Score (Log Loss):", study.best_value)


print("Retraining on the full dataset with best hyperparameters...")
final_br = BinaryRelevance(
    classifier=XGBClassifier(use_label_encoder=False, eval_metric='logloss',
        **best_params),
    require_dense=[False, True]
)
final_br.fit(X, Y)

test_probs = []
for i, clf in enumerate(final_br.classifiers_):
    p = clf.predict_proba(test_features.values)[:,1]
    test_probs.append(p)
test_probs = np.column_stack(test_probs)

submission_df = sample_submission.copy()
submission_df.loc[:, MoA_columns] = test_probs
submission_df.to_csv('submission.csv')
print("Submission file created: submission.csv")


#### ECC #####

import pandas as pd
import numpy as np
import xgboost as xgb
from sklearn.model_selection import train_test_split
from skmultilearn.problem_transform import ClassifierChain
from xgboost import XGBClassifier
```

```python
from sklearn.metrics import log_loss
import sys


train_features = pd.read_csv('train_features.csv', index_col=0)
train_targets_scored = pd.read_csv('train_targets_scored.csv', index_col=0)
test_features = pd.read_csv('test_features.csv', index_col=0)
sample_submission = pd.read_csv('sample_submission.csv', index_col=0)

MoA_columns = train_targets_scored.columns.tolist()

def preprocess_features(df):
    df['cp_dose_0'] = (df['cp_dose']=='D1').astype(int)
    df['cp_dose_1'] = (df['cp_dose']=='D2').astype(int)

    df['cp_time_24'] = (df['cp_time']==24).astype(int)
    df['cp_time_48'] = (df['cp_time']==48).astype(int)
    df['cp_time_72'] = (df['cp_time']==72).astype(int)

    df = df.drop(['cp_time','cp_dose','cp_type'], axis=1, errors='ignore')
    return df

train_features = preprocess_features(train_features)
test_features = preprocess_features(test_features)

X = train_features.values
Y = train_targets_scored.values

X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.2,
    random_state=42)
print(f"X_train: {X_train.shape}, X_val: {X_val.shape}, Y_train: {Y_train.
    shape}, Y_val: {Y_val.shape}")

def build_ecc_ensemble(base_classifier, ensemble_size, X_train, Y_train):

    chains = []
    num_labels = Y_train.shape[1]
    for _ in range(ensemble_size):
        label_order = np.random.permutation(num_labels)
        chain = ClassifierChain(
            classifier=base_classifier,
            order=label_order,
            require_dense=[False, True]
        )
        chain.fit(X_train, Y_train)
        chains.append(chain)
    return chains

def ecc_predict_proba(chains, X_data):

    chain_probs_list = []
    for chain in chains:
        valX = X_data.copy()
        chain_probs = []
        for clf in chain.classifiers_:
            proba = clf.predict_proba(valX, validate_features=False)[:,1]
            chain_probs.append(proba)

            label_pred = (proba >= 0.5).astype(int).reshape(-1,1)
            valX = np.hstack([valX, label_pred])

        chain_probs = np.column_stack(chain_probs)
```

```
        chain_probs_list.append(chain_probs)
    avg_probs = np.mean(chain_probs_list, axis=0)
    return avg_probs

xgb_params = {
    'use_label_encoder': False,
    'eval_metric': 'logloss',
    'objective': 'binary:logistic',
    'max_depth': 5,
    'learning_rate': 0.02205645796190082,
    'n_estimators': 311,
    'subsample': 0.9923903934121262,
    'colsample_bytree': 0.7206271866585845,
    'tree_method':'gpu_hist',
    'device':'cuda'
}

chains = build_ecc_ensemble(XGBClassifier(**xgb_params), ensemble_size=3,
    X_train=X_train, Y_train=Y_train)

val_probs = ecc_predict_proba(chains, X_val)

score = 0.0
for i in range(Y_val.shape[1]):
    score += log_loss(Y_val[:, i], val_probs[:, i], labels=[0,1])
score /= Y_val.shape[1]
print("Validation Log Loss:", score)


final_chains = build_ecc_ensemble(XGBClassifier(**xgb_params), ensemble_size
    =5, X_train=X, Y_train=Y)


test_probs = ecc_predict_proba(final_chains, test_features.values)

submission_df = sample_submission.copy()
submission_df.loc[:, MoA_columns] = test_probs
submission_df.to_csv('submission.csv', index=False)
print("Submission file created: submission.csv")


###### R'S MLDR #######

library(mldr)
library(readr)
filtered_training <- read_csv("filtered_training.csv")
View(filtered_training)
mlsmote <- read_csv('MLSMOTE1.csv')
mlsmote2 <- read_csv('SMOTE5.csv')
mlsmote3 <- read_csv('SMOTE6.csv')
mlsmote4 <- read_csv('SMOTE7.csv')
mlsmote5 <- read_csv('SMOTE8.csv')


str(filtered_training)


label_columns <- c("5-alpha_reductase_inhibitor", "11-beta-hsd1_inhibitor", "
    acat_inhibitor", "acetylcholine_receptor_agonist", "
    acetylcholine_receptor_antagonist", "acetylcholinesterase_inhibitor", "
    adenosine_receptor_agonist", "adenosine_receptor_antagonist", "
    adenylyl_cyclase_activator", "adrenergic_receptor_agonist", "
```

```
adrenergic_receptor_antagonist", "akt_inhibitor", "
aldehyde_dehydrogenase_inhibitor", "alk_inhibitor", "ampk_activator", "
analgesic", "androgen_receptor_agonist", "androgen_receptor_antagonist", "
anesthetic_-_local",
                "angiogenesis_inhibitor", "angiotensin_receptor_antagonist"
                    , "anti-inflammatory", "antiarrhythmic", "antibiotic",
                    "anticonvulsant", "antifungal", "antihistamine", "
                    antimalarial", "antioxidant", "antiprotozoal", "
                    antiviral", "apoptosis_stimulant", "aromatase_inhibitor
                    ", "atm_kinase_inhibitor", "atp-
                    sensitive_potassium_channel_antagonist", "
                    atp_synthase_inhibitor", "atpase_inhibitor", "
                    atr_kinase_inhibitor", "aurora_kinase_inhibitor", "
                    autotaxin_inhibitor", "
                    bacterial_30s_ribosomal_subunit_inhibitor", "
                    bacterial_50s_ribosomal_subunit_inhibitor", "
                    bacterial_antifolate", "
                    bacterial_cell_wall_synthesis_inhibitor", "
                    bacterial_dna_gyrase_inhibitor", "
                    bacterial_dna_inhibitor", "
                    bacterial_membrane_integrity_inhibitor", "bcl_inhibitor
                    ", "bcr-abl_inhibitor", "
                    benzodiazepine_receptor_agonist", "
                    beta_amyloid_inhibitor", "bromodomain_inhibitor", "
                    btk_inhibitor", "calcineurin_inhibitor", "
                    calcium_channel_blocker", "cannabinoid_receptor_agonist
                    ", "cannabinoid_receptor_antagonist", "
                    carbonic_anhydrase_inhibitor", "casein_kinase_inhibitor
                    ", "caspase_activator", "
                    catechol_o_methyltransferase_inhibitor", "
                    cc_chemokine_receptor_antagonist", "
                    cck_receptor_antagonist", "cdk_inhibitor", "
                    chelating_agent", "chk_inhibitor", "
                    chloride_channel_blocker", "cholesterol_inhibitor", "
                    cholinergic_receptor_antagonist", "
                    coagulation_factor_inhibitor", "corticosteroid_agonist"
                    , "cyclooxygenase_inhibitor", "
                    cytochrome_p450_inhibitor", "
                    dihydrofolate_reductase_inhibitor", "
                    dipeptidyl_peptidase_inhibitor", "diuretic", "
                    dna_alkylating_agent", "dna_inhibitor", "
                    dopamine_receptor_agonist", "
                    dopamine_receptor_antagonist", "egfr_inhibitor", "
                    elastase_inhibitor", "erbb2_inhibitor", "
                    estrogen_receptor_agonist", "
                    estrogen_receptor_antagonist", "faah_inhibitor", "
                    farnesyltransferase_inhibitor", "
                    fatty_acid_receptor_agonist", "fgfr_inhibitor", "
                    flt3_inhibitor", "focal_adhesion_kinase_inhibitor", "
                    free_radical_scavenger", "
                    fungal_squalene_epoxidase_inhibitor", "
                    gaba_receptor_agonist", "gaba_receptor_antagonist",
                "gamma_secretase_inhibitor", "
                    glucocorticoid_receptor_agonist", "glutamate_inhibitor"
                    , "glutamate_receptor_agonist", "
                    glutamate_receptor_antagonist", "
                    gonadotropin_receptor_agonist", "gsk_inhibitor", "
                    hcv_inhibitor", "hdac_inhibitor", "
                    histamine_receptor_agonist", "
                    histamine_receptor_antagonist", "
                    histone_lysine_demethylase_inhibitor", "
                    histone_lysine_methyltransferase_inhibitor", "
```

```
                    hiv_inhibitor", "hmgcr_inhibitor", "hsp_inhibitor", "
                    igf-1_inhibitor", "ikk_inhibitor",
                "imidazoline_receptor_agonist", "immunosuppressant", "
                    insulin_secretagogue", "insulin_sensitizer", "
                    integrin_inhibitor", "jak_inhibitor", "kit_inhibitor",
                    "laxative", "leukotriene_inhibitor", "
                    leukotriene_receptor_antagonist", "lipase_inhibitor", "
                    lipoxygenase_inhibitor", "lxr_agonist", "mdm_inhibitor"
                    , "mek_inhibitor", "membrane_integrity_inhibitor", "
                    mineralocorticoid_receptor_antagonist", "
                    monoacylglycerol_lipase_inhibitor", "
                    monoamine_oxidase_inhibitor", "
                    monopolar_spindle_1_kinase_inhibitor",
                "mtor_inhibitor", "mucolytic_agent", "
                    neuropeptide_receptor_antagonist", "nfkb_inhibitor", "
                    nicotinic_receptor_agonist", "nitric_oxide_donor", "
                    nitric_oxide_production_inhibitor", "
                    nitric_oxide_synthase_inhibitor", "
                    norepinephrine_reuptake_inhibitor", "nrf2_activator", "
                    opioid_receptor_agonist", "opioid_receptor_antagonist",
                     "orexin_receptor_antagonist", "p38_mapk_inhibitor", "p
                    -glycoprotein_inhibitor", "parp_inhibitor",
                "pdgfr_inhibitor", "pdk_inhibitor", "
                    phosphodiesterase_inhibitor", "phospholipase_inhibitor"
                    , "pi3k_inhibitor", "pkc_inhibitor", "
                    potassium_channel_activator", "
                    potassium_channel_antagonist", "ppar_receptor_agonist",
                     "ppar_receptor_antagonist", "
                    progesterone_receptor_agonist", "
                    progesterone_receptor_antagonist", "
                    prostaglandin_inhibitor", "
                    prostanoid_receptor_antagonist", "proteasome_inhibitor"
                    , "protein_kinase_inhibitor", "
                    protein_phosphatase_inhibitor", "
                    protein_synthesis_inhibitor",
                "protein_tyrosine_kinase_inhibitor", "radiopaque_medium", "
                    raf_inhibitor", "ras_gtpase_inhibitor", "
                    retinoid_receptor_agonist", "
                    retinoid_receptor_antagonist", "
                    rho_associated_kinase_inhibitor", "
                    ribonucleoside_reductase_inhibitor", "
                    rna_polymerase_inhibitor", "serotonin_receptor_agonist"
                    , "serotonin_receptor_antagonist", "
                    serotonin_reuptake_inhibitor", "sigma_receptor_agonist"
                    , "sigma_receptor_antagonist", "
                    smoothened_receptor_antagonist", "
                    sodium_channel_inhibitor", "
                    sphingosine_receptor_agonist",
                "src_inhibitor", "steroid", "syk_inhibitor", "
                    tachykinin_antagonist", "tgf-beta_receptor_inhibitor",
                    "thrombin_inhibitor", "thymidylate_synthase_inhibitor",
                     "tlr_agonist", "tlr_antagonist", "tnf_inhibitor", "
                    topoisomerase_inhibitor", "
                    transient_receptor_potential_channel_antagonist", "
                    tropomyosin_receptor_kinase_inhibitor", "trpv_agonist",
                     "trpv_antagonist", "tubulin_inhibitor", "
                    tyrosine_kinase_inhibitor", "
                    ubiquitin_specific_protease_inhibitor", "
                    vegfr_inhibitor", "vitamin_b", "
                    vitamin_d_receptor_agonist", "wnt_inhibitor")
```

```r

library(janitor)


dataset <- janitor::clean_names(filtered_training)
dataset1 <- janitor::clean_names(mlsmote)
dataset2 <- janitor::clean_names(mlsmote2)
dataset3 <- janitor::clean_names(mlsmote3)
dataset4 <- janitor::clean_names(mlsmote4)

dataset5 <- janitor::clean_names(mlsmote5)

dataset_no_id <- dataset[, !names(dataset) %in% "sig_id"]
dataset_no_id <- dataset1[, !names(dataset1) %in% "sig_id"]
dataset_no_id <- dataset2[, !names(dataset2) %in% "sig_id"]
dataset_no_id <- dataset3[, !names(dataset3) %in% "sig_id"]
dataset_no_id <- dataset4[, !names(dataset4) %in% "sig_id"]
dataset_no_id <- dataset5[, !names(dataset5) %in% "sig_id"]

colSums(MoA$dataset[, MoA$labels$index])


clean_label_columns <- janitor::make_clean_names(label_columns)





label_indices <- which(names(dataset_no_id) %in% clean_label_columns)



MoA <- mldr_from_dataframe(dataset_no_id, labelIndices = label_indices, name = "MoA")
MoA1 <- mldr_from_dataframe(dataset_no_id, labelIndices = label_indices, name = "MoA1")
MoA2 <- mldr_from_dataframe(dataset_no_id, labelIndices = label_indices, name = "MoA2")
MoA3 <- mldr_from_dataframe(dataset_no_id, labelIndices = label_indices, name = "MoA3")
MoA4 <- mldr_from_dataframe(dataset_no_id, labelIndices = label_indices, name = "MoA4")

MoA5 <- mldr_from_dataframe(dataset_no_id, labelIndices = label_indices, name = "MoA5")
##################################################################

summary(MoA4)




write_arff(mld, "output_dataset.arff")

install.packages("xml2")
library(xml2)
# ##############################################################################################

####COVNERTING TEST DATA TOO##########################

library(mldr)
```

```
library(readr)
library(janitor)

test_features <- read_csv("test_features.csv")

test_dataset <- janitor::clean_names(test_features)


test_dataset_no_id <- test_dataset[, !names(test_dataset) %in% "sig_id"]
h cleaned names
clean_label_columns <- janitor::make_clean_names(label_columns)


label_indices <- which(names(test_dataset_no_id) %in% clean_label_columns)


MoA_test <- mldr_from_dataframe(test_dataset_no_id, labelIndices =
    label_indices, name = "MoA_test")

write_arff(MoA_test, "test_dataset.arff")


cat("Test dataset successfully converted to ARFF format: test_dataset.arff\n")


##################################################


attributes <- mld$attributes
labels <- mld$labels$name

library(mldr)
library(xml2)


print(mld)


root <- xml_new_root("dataset")

xml_add_child(root, "name", mld$name)
xml_add_child(root, "instances", as.character(mld$measures$num.instances))
xml_add_child(root, "features", as.character(mld$measures$num.attributes))
xml_add_child(root, "labels", as.character(mld$measures$num.labels))

features <- xml_add_child(root, "features")
for (feature in mld$attributes$name) {
  xml_add_child(features, "feature", feature)
}

labels <- xml_add_child(root, "labels")
for (label in mld$labels$name) {
  xml_add_child(labels, "label", label)
}
write_xml(root, "output_dataset.xml")


print(MoA1$measures)


print(mld$labelsets)

```

```

print(mld$measures$cardinality)


library(ggplot2)


label_freq <- data.frame(
  Label = MoA1$labels$name,
  Frequency = MoA1$labels$frequency
)

print(label_freq)

##################################################

summary(MoA)
summary(MoA2)

mld$labels


plot(MoA, type = "LH")

layout(matrix(c(1,1,6,1,1,3,5,5,3,2,4,7), 4, 3, byrow = TRUE)
plot(MoA, type = "LC", labelCount = 100)
png("circular_plot1.png", width = 1000, height = 1000, res = 400)
plot(MoA2, type = "LB")
dev.off()
plot(MoA2, type = "CH")
plot(MoA2, type = "LSB")
png("pie.png", width = 600, height = 600, res = 300)
dev.off()
plot(MoA2, type = "AT")
plot(MoA2, type = "LB")

plot(MoA2, type = "LSH", xlim = c(0, 800))

plot(MoA, type = "LSH", xlim = c(0, 1000))
axis(1, at = seq(0, 1000, by = 100))





head(labelset_freq_df)


concurrenceReport(MoA)
#######################

##### MLSMOTE #####

import numpy as np
from sklearn.neighbors import NearestNeighbors
from collections import Counter
import pandas as pd
import argparse

def calculate_IR(Y):

    n_samples, n_labels = Y.shape
```

```python
    IRs = []
    for j in range(n_labels):
        label_count = np.sum(Y[:, j])
        if label_count == 0:

            IRs.append(0.0)
        else:
            IR = (n_samples - label_count) / label_count
            IRs.append(IR)
    return IRs, np.mean(IRs)

def most_frequent(values):
    return Counter(values).most_common(1)[0][0]

def mlsmote(X, Y, k=5,
    if random_state is not None:
        np.random.seed(random_state)

    n_samples, n_features = X.shape
    _, n_labels = Y.shape

    IRs, MeanIR = calculate_IR(Y)

    X_res = X.copy()
    Y_res = Y.copy()


    for j in range(n_labels):
        if IRs[j] > MeanIR:

            minBag_indices = np.where(Y[:, j] == 1)[0]
            if len(minBag_indices) == 0:
                continue


            X_min = X[minBag_indices]
            Y_min = Y[minBag_indices]

            if len(minBag_indices) <= k:

            nbrs = NearestNeighbors(n_neighbors=k+1).fit(X_min)
            distances, indices = nbrs.kneighbors(X_min)

            for i, idx in enumerate(minBag_indices):

                neighbor_indices = indices[i, 1:]  # skip i-th itself
                neighbors_X = X_min[neighbor_indices]
                neighbors_Y = Y_min[neighbor_indices]

                ref_index = np.random.choice(neighbor_indices)
                refNeigh = X[ minBag_indices[np.where(neighbor_indices==
                    ref_index)[0][0]] ]

                synthX, synthY = create_synthetic_sample(
                    X[idx], Y[idx],
                    refNeigh, neighbors_X, neighbors_Y
                )

                X_res = np.vstack([X_res, synthX.reshape(1, -1)])
                Y_res = np.vstack([Y_res, synthY.reshape(1, -1)])

    return X_res, Y_res
```

```python

def create_synthetic_sample(sampleX, sampleY, refNeighX, neighborsX, neighborsY):

    synthX = np.zeros_like(sampleX)
    n_neighbors = neighborsX.shape[0]


    for f in range(len(sampleX)):
        if np.issubdtype(sampleX[f].dtype, np.number):
            diff = refNeighX[f] - sampleX[f]
            offset = diff * np.random.rand()
            value = sampleX[f] + offset
        else:
            cat_values = np.concatenate([[sampleX[f]], neighborsX[:, f]])
            value = most_frequent(cat_values)
        synthX[f] = value

    combined_labels = np.vstack([sampleY, neighborsY])
    label_counts = np.sum(combined_labels, axis=0)
    threshold = (n_neighbors + 1) / 2
    synthY = (label_counts > threshold).astype(int)

    return synthX, synthY


def main():
    parser = argparse.ArgumentParser(description="MLSMOTE: Multi-Label Synthetic Oversampling")
    parser.add_argument("input_file", help="Path to input CSV file (features and labels).")
    parser.add_argument("output_file", help="Path to save the resampled CSV file.")
    parser.add_argument("--k", type=int, default=5, help="Number of nearest neighbors.")
    parser.add_argument("--random_state", type=int, default=42, help="Random seed for reproducibility.")
    args = parser.parse_args()


    data = pd.read_csv(args.input_file, index_col=0)
    print("Initial Columns:", data.columns)

    n_labels = 206
    labels = data.iloc[:, -n_labels:]
    features = data.iloc[:, :-n_labels]

    X = features.values
    Y = labels.values

    print("Initial Feature Shape:", X.shape)
    print("Initial Label Shape:", Y.shape)

    X_res, Y_res = mlsmote(X, Y, k=args.k, random_state=args.random_state)

    print("Resampled Feature Shape:", X_res.shape)
    print("Resampled Label Shape:", Y_res.shape)


    resampled_data = np.hstack((X_res, Y_res))

    output_columns = features.columns.tolist() + labels.columns.tolist()
```

```
    assert len(output_columns) == resampled_data.shape[1], "Mismatch in column count!"

    output_df = pd.DataFrame(resampled_data, columns=output_columns)
    output_df.to_csv(args.output_file, index=False)
    print(f"Resampled data saved to: {args.output_file}")

    reloaded_data = pd.read_csv(args.output_file)
    print("Reloaded Data Shape:", reloaded_data.shape)
    assert reloaded_data.shape[1] == len(output_columns), "Column count mismatch after saving!"




if __name__ == "__main__":
    main()


##############################

##### LP + IG + LDA + BR + XGBOOST####

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from skmultilearn.problem_transform import LabelPowerset, BinaryRelevance
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.feature_selection import mutual_info_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import KFold
from sklearn.metrics import f1_score
from xgboost import XGBClassifier

.
lp = LabelPowerset(
    classifier=XGBClassifier(use_label_encoder=False, eval_metric='logloss', n_estimators=10, max_depth=3),
    require_dense=[False, True]
)
lp.fit(X_train, Y_train)
Y_lp_train = lp.transform(X_train)
Y_lp_train = Y_lp_train.toarray() if hasattr(Y_lp_train, 'toarray') else Y_lp_train
if Y_lp_train.ndim > 1 and Y_lp_train.shape[1] == 1:
    Y_lp_train = Y_lp_train.ravel()


ig_scores = mutual_info_classif(X_train if isinstance(X_train, np.ndarray) else X_train.values, Y_lp_train)
ig_ranked_features = np.argsort(ig_scores)[::-1]


top_features_ig = ig_ranked_features[:100]


if isinstance(X_train, pd.DataFrame):
    X_train_ig = X_train.iloc[:, top_features_ig].values
else:
    X_train_ig = X_train[:, top_features_ig]


```

```python
num_classes = len(np.unique(Y_lp_train))
lda = LDA(n_components=min(num_classes - 1, 10))
X_train_reduced = lda.fit_transform(X_train_ig, Y_lp_train)


if isinstance(X_test, pd.DataFrame):
    X_test_ig = X_test.iloc[:, top_features_ig].values
else:
    X_test_ig = X_test[:, top_features_ig]

X_test_reduced = lda.transform(X_test_ig)



####

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from skmultilearn.problem_transform import LabelPowerset, BinaryRelevance
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from xgboost import XGBClassifier
from sklearn.model_selection import KFold
from sklearn.metrics import f1_score, log_loss

feature_names = X.columns if isinstance(X, pd.DataFrame) else np.arange(X.
    shape[1]).astype(str)

best_params = {
    'max_depth': 5,
    'learning_rate': 0.02205645796190082,
    'n_estimators': 311,
    'subsample': 0.9923903934121262,
    'colsample_bytree': 0.7206271866585845,
    'use_label_encoder': False,
    'eval_metric': 'logloss',
    'objective': 'binary:logistic',
    'device': 'cuda'
}

br_classifier = BinaryRelevance(
    classifier=XGBClassifier(**best_params),
    require_dense=[False, True]
)


kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = []
for train_idx, val_idx in kf.split(X_train_reduced, Y_lp_train):
    X_tr, X_val_ = X_train_reduced[train_idx], X_train_reduced[val_idx]
    Y_tr, Y_val_ = Y_train[train_idx], Y_train[val_idx]
    br_classifier.fit(X_tr, Y_tr)
    Y_pred = br_classifier.predict(X_val_)
    Y_pred = Y_pred.toarray() if hasattr(Y_pred, 'toarray') else Y_pred
    score = f1_score(Y_val_, Y_pred, average='macro')
    cv_scores.append(score)

print("CV Mean F1-score with BR:", np.mean(cv_scores))

br_classifier.fit(X_train_reduced, Y_train)
Y_test_pred = br_classifier.predict(X_test_reduced)
```

```python
Y_test_pred = Y_test_pred.toarray() if hasattr(Y_test_pred, 'toarray') else Y_test_pred
test_score = f1_score(Y_test, Y_test_pred, average='macro')
print("Test F1-score with BR:", test_score)

val_probs_final = br_classifier.predict_proba(X_test_reduced)


adjusted_val_probs = []
for p in val_probs_final:
    if hasattr(p, "toarray"):
        p = p.toarray()


    if p.ndim == 1:

        p = np.column_stack([1-p, p])
    elif p.shape[1] == 1:

        p = np.column_stack([1-p[:,0], p[:,0]])

    adjusted_val_probs.append(p)

val_probs_final_stack = np.column_stack([p[:,1] for p in adjusted_val_probs])

final_score = 0.0
for i in range(Y_test.shape[1]):
    final_score += log_loss(Y_test[:, i], val_probs_final_stack[:, i], labels=[0,1])
final_score /= Y_test.shape[1]
print("Final Validation Log Loss:", final_score)
```